

HALT: Quantized Reasoning Models Don’t Stop, and a Harness-Applied Limit on Thinking Repairs Them

Aditya Karnam Gururaj Rao
adityakarnam.grao@gmail.com

Arjun Jaggi
arjunjaggi@gmail.com

Abstract

Quantizing Ornith-1.5-9B, a 9B hybrid-attention reasoning model, to 4.7 bits costs 4.6 points of MMLU and 3.7 of HumanEval against its bf16 parent. We set out to recover that loss by distillation and, over twelve days on a single 64GB Apple M4 Pro, tried nine things. The first worked: two-teacher sequence-level distillation with rejection sampling on verified traces, trained in bf16 and quantized last, gives a model at identical size and bit width that scores +5.5 MMLU and +3.0 HumanEval over the quantized baseline, beats the $1.8\times$ larger 8-bit build on both, and matches its own 35B teacher on MMLU—while generating 15–33% fewer tokens. The next seven did not: varying data mixture, volume, length distribution, training objective, filter, adapter capacity, and on-policy prompt selection all failed to improve on it, and a randomised control showed that a second training pass from the best checkpoint costs 1–4 points regardless of what it trains on. Diagnosing *why* the first run worked turned out to be the useful result: most of what quantization costs this model is not reasoning ability but the ability to *stop*. The 4-bit build fails to reach an answer on 24% of HumanEval problems its parent finishes, 22 of the 28 problems distillation recovered were budget exhaustions, and the effect reproduces as wall clock on a harness with no truncation penalty. The ninth attempt followed from that diagnosis and needed no training at all. HALT, a Harness-Applied Limit on Thinking—close the reasoning block at the decode budget and let the model answer—recovers +7.3 points of HumanEval at a 2,048-token budget and +3.6 at 4,096 on the same quantized weights, and a model forced to stop at 2,048 tokens outperforms the same model allowed 4,096. We conclude that termination is a decoding-time property before it is a weights property, that any benchmark of a reasoning model that does not report its truncation rate is partly reporting its own token budget, and we release the pipeline, every checkpoint, per-item evaluation records and a 45-section decision log. Along the way we contribute a chunkwise-parallel training path for gated DeltaNet layers in MLX ($24\times$ memory, $135\times$ speed at 4,096 tokens), now upstream.

1 Introduction

Post-training quantization is how a reasoning model reaches consumer hardware, and its cost is customarily reported as a few points of benchmark accuracy. This paper asks what those points *are*. It began as a distillation project—recover the quantization loss by teaching the 4-bit model from larger ones—and it succeeded on the first attempt. It then spent seven further runs failing to improve on that success, each moving one variable with a prediction recorded before measurement, until the accumulated negative results and the per-item evaluation records forced a different question: not *what did the successful run teach the model*, but *what was broken that it fixed*.

The answer is that the quantized model does not stop. On a strict harness at a 4,096-token budget, stock oQ4 Ornith-1.5-9B never reaches an answer on 39 of 164 HumanEval problems; its

bf16 parent truncates on 15. The distilled model truncates on 19, and 22 of the 28 problems it gains over stock are ones where stock ran out of budget mid-reasoning. Along the quantization ladder, the 3-bit build takes $2.2\times$ as long as the 4-bit build to answer the same questions while scoring 11 points lower, despite having fewer bits to multiply. The same bf16 weights score 82.3% or 88.4% on HumanEval depending only on whether the budget is 2,048 or 4,096 tokens.

Once the failure is named, the cheapest repair is obvious: stop the model for it. We call the result HALT, a Harness-Applied Limit on Thinking: the closing half of budget forcing [15], applied as a two-phase decoder that closes the reasoning block when the budget is nearly spent and lets the model commit. It is a decoder change, not a training run. It is the ninth thing we tried and, per token of budget, the best.

Contributions.

1. A training-side repair (§3–4): two-teacher sequence-level distillation with rejection sampling, +5.5 MMLU and +3.0 HumanEval at identical size and bit width.
2. A diagnosis (§5): quantization damages termination more than reasoning, evidenced six ways, including in the 35B teacher.
3. A decoding-side repair (§6): HALT, a harness-applied limit on thinking, on the same weights, with the finding that stopping *earlier* than the model would stop itself improves accuracy.
4. Eight controlled negative results (§7), including a randomised control showing that continued training from the best checkpoint is itself harmful, and evidence that the teacher cannot supply the one behaviour the student lacks.
5. An engineering result (§8): a differentiable chunkwise training path for gated DeltaNet in MLX, contributed upstream.

2 Setup

Student. Ornith-1.5-9B is a 32-layer hybrid-attention reasoning model with gated DeltaNet (GDN) linear-attention layers [19], a 248,044-token vocabulary and a thinking-mode chat template that opens a `<think>` block before every answer. The deployed quantization, oQ4, is 4-bit affine at group size 64 with 120 modules (predominantly linear-attention projections) promoted to 5, 6 or 8 bits, averaging 4.721 bits per weight. The promotion map is recoverable from the checkpoint’s configuration and we replay it exactly, so every quantized comparison in this paper is like-for-like rather than “our 4-bit against theirs”.

Teachers. Two 35B-A3B mixture-of-experts models at 4 bits, routed by measured strength rather than reputation (Table 1). Qwen3.6-35B-A3B is 6.3 points stronger on MMLU and teaches knowledge and truthfulness; Ornith-1.5-35B ties it on HumanEval while sampling $4.6\times$ faster and teaches code. All three models share an index-identical vocabulary (verified: vocabulary and merges equal; Qwen’s pre-tokenizer regex differs, affecting text encoding but not token identity), which is what makes logit-level distillation (§7.2) possible at all.

Table 1: Teachers, measured under the same protocol as the student.

Teacher	MMLU	TruthfulQA	HumanEval	Sampling speed
Qwen3.6-35B-A3B, 4-bit	89.3	89.2	93.3	1.0×
Ornith-1.5-35B-A3B, 4-bit	83.0	86.4	93.3	4.6×

Benchmarks and protocol. MMLU [8] on a fixed seeded 1,000-question sample of 14,042; TruthfulQA MC1 [13], all 817; HumanEval [3], all 164. Thinking enabled, greedy decoding. Two harnesses were used. The oMLX server is the deployment target and the source of every headline number. Our own harness (`odistil eval`) records per-item truncation, token counts and wall clock, and is what the mechanism analysis rests on. The two disagree by up to 15 points on the *stock* model’s HumanEval, because oMLX recovers an answer from output ours scores as unfinished; they agree on every direction, and no table mixes them.

Evaluation hygiene. TruthfulQA lists the correct choice first in all 817 items. Presented in dataset order, every gold answer is “A”, and a model that always answers A scores 100%; an early version of our harness did exactly this and reported a metric that was measuring position bias. We shuffle choices under a per-item seed. Model-generated code runs only in a subprocess with a temporary working directory and a hard timeout. Training data is decontaminated against all three evaluation sets by 13-gram overlap.

Hardware. One Apple M4 Pro with 64 GB of unified memory. A full fine-tune of a 9B does not fit (≈ 18 GB of weights plus ≈ 72 GB of fp32 AdamW state); LoRA [10] over the bf16 base does. Only one model is resident at a time, which shapes every stage of the pipeline.

3 Method: the run that worked

Sequence-level distillation with rejection sampling. Following Kim and Rush [11], the student learns from teacher-generated sequences rather than teacher logits. For each prompt in a pool drawn from MMLU auxiliary-train, ARC, OpenBookQA, TriviaQA, SciQ and MBPP, the owning teacher generates one reasoning trace with thinking enabled. The trace is kept only if it verifies: final letter matches gold (multiple choice), normalised answer matches a gold alias (open QA), or the extracted program passes the dataset’s own tests (code). Qwen scores 89.3% on MMLU, so roughly one unfiltered knowledge trace in nine would be a confidently argued wrong answer. Traces exceeding the training length cap are dropped rather than truncated: a truncated target teaches the model to stop mid-answer, which is the opposite of what §5 shows it needs.

Train in bf16, quantize last. LoRA at rank 32 over the top 16 of 32 layers ($\alpha = 16$, dropout 0.05), 3,200 steps at batch size 1, learning rate 3×10^{-5} with cosine decay and 150 warmup steps, 1,024-token cap. Adapters are fused into a bf16 checkpoint, which is then quantized with the replayed oQ4 map, so the quantizer starts from an improved checkpoint instead of compounding two lossy steps. QLoRA [5] over the quantized base was measured and rejected: 49.0 GB peak against bf16’s 47.3 and 26 s per iteration against 10, because dequantising every step costs more than the smaller weights save.

Table 2: Main result. All rows measured on the oMLX server under the same protocol (MMLU: fixed 1,000-question sample; TruthfulQA: all 817; HumanEval: all 164; thinking on; greedy). Size is the on-disk checkpoint. The distilled model is quantized with the identical oQ4 map as the stock build.

Model	GB	MMLU	TruthfulQA	HumanEval	HumanEval s
Ornith-1.5-9B oQ3	3.9	66.6	67.6	72.0	5,585
Ornith-1.5-9B oQ4 (stock)	4.9	78.0	80.7	87.8	4,171
Ornith-1.5-9B oQ8	8.9	83.1	80.7	88.4	4,797
Ornith-1.5-9B bf16	17	82.6	80.5	91.5	9,994
distilled oQ4 (v1, ours)	4.9	83.5	79.0	90.8	2,786
Ornith-1.5-35B-A3B 4-bit (teacher)	18	83.0	86.4	93.3	1,043
Qwen3.6-35B-A3B 4-bit (teacher)	19	89.3	89.2	93.3	4,804

Table 3: Stock and distilled oQ4 on our harness, HumanEval, 4,096-token budget.

	Stock oQ4	Distilled oQ4
HumanEval	72.0%	84.1%
Never reached an answer	39 / 164 (24%)	19 / 164 (12%)
Mean generation (tokens)	1,721	1,414

Mixture. 40% knowledge, 25% truthfulness, 35% code by sample count, sized from whichever domain is most over-supplied so that scarcity in one domain never discards verified data in another. The run we call v1 trained on 1,639 samples with 33 held out.

4 Main result

Table 2 gives the result. Against the model it replaces, the distilled oQ4 build is **+5.5 MMLU and +3.0 HumanEval at identical size and bit width**, and -1.7 TruthfulQA. It beats the $1.8\times$ larger 8-bit build on both gains, edges the $3.5\times$ larger bf16 parent on MMLU, and matches its own 35B code teacher on MMLU. It is also faster: -33% wall clock on HumanEval and -15% on MMLU against stock—the same weights per token, fewer tokens per answer. The TruthfulQA regression is real and is analysed in §7.4.

This was the first run. Section 7 describes the seven that followed it and failed. Section 5 explains why the first one worked, which is what eventually pointed at the ninth.

5 The mechanism: quantization breaks stopping

5.1 Truncation explains the gain

On our harness at a 4,096-token HumanEval budget, with both models quantized by the identical map (Table 3), the distilled model gained 28 problems over stock and lost 8. **Twenty-two of the 28 gains were problems on which the stock model ran out of budget mid-reasoning.** The gain is not mostly new capability; it is mostly finishing. Truncation falls on every benchmark at once: MMLU $24\rightarrow 10$, TruthfulQA $27\rightarrow 20$, HumanEval $39\rightarrow 19$.

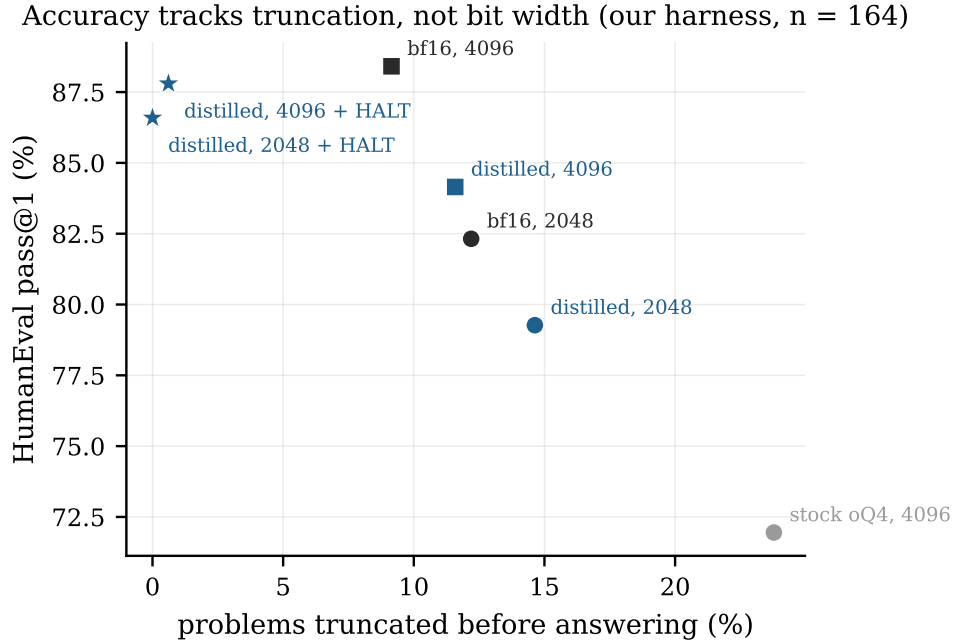


Figure 1: The paper in one plot. Every HumanEval measurement our harness has made on all 164 items, placed by the fraction of problems on which the model never reached an answer. Bit width does not order the points; truncation does. Stars are HALT runs on the same weights as the circle or square of the same colour.

5.2 It is not a scoring artefact

The natural objection is that a harness which penalises truncation will of course find truncation explains its scores. The effect reproduces on a harness that does not. oMLX recovers an answer from output ours scores as unfinished, scoring stock oQ4 at 87.8% where ours gives 72.0%. On that forgiving harness the same weights show the effect as *wall clock*: -33% on HumanEval, -15% on MMLU ($+26\%$ on TruthfulQA, where the distilled model lost). A model that reaches its answer a third sooner on the benchmark where it improved, measured by a harness with no truncation penalty at all, is a property of the model and not of the scoring.

5.3 The quantization ladder

Table 4 is the strongest single piece of evidence and the one with the clearest caveat. The 3-bit build has fewer bits than the 4-bit build, so its per-token throughput should be at least as high; it takes $2.23\times$ as long for the same 1,000 questions while scoring 11.4 points lower. The extra time is extra tokens: it is reasoning at much greater length and arriving somewhere worse. These are wall-clock totals, because the external harness does not emit per-item token counts; the inference is strong but not airtight.

5.4 Budget sensitivity on unchanged weights

The bf16 base on our harness, only the budget changed: 82.3% HumanEval with 20 truncations at 2,048 tokens; 88.4% with 15 at 4,096. Six points of “model quality” that are entirely an evaluation

Table 4: The quantization ladder on the same 1,000 MMLU questions (oMLX). oQ3 has fewer bits than oQ4 and should be at least as fast per token; it takes $2.23\times$ as long. bf16’s $2.20\times$ is unquantized compute being slower per token, a different effect. Wall-clock totals, not token counts.

Build	MMLU	Seconds	vs oQ4
oQ3	66.6	34,085	$2.23\times$
oQ4	78.0	15,262	$1.00\times$
oQ8	83.1	16,434	$1.08\times$
bf16	82.6	33,507	$2.20\times$
distilled oQ4	83.5	13,007	$0.85\times$

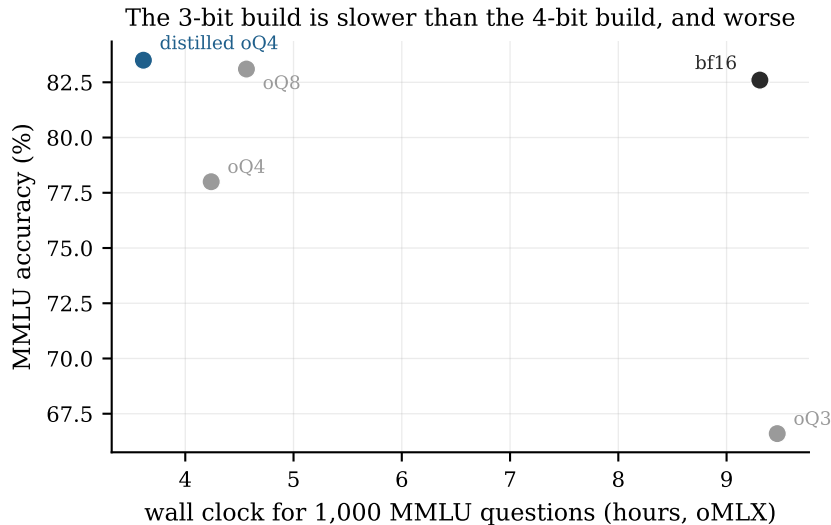


Figure 2: The quantization ladder (Table 4) as accuracy against wall clock for the same 1,000 MMLU questions. The 3-bit build has fewer bits to multiply than the 4-bit build and takes $2.2\times$ as long, because it generates more tokens; the distilled 4-bit build is both the most accurate and the fastest.

parameter—and 15 items still truncate at the larger budget, so the number never reaches zero, it just stops mattering as much.

5.5 Multiple choice hides it

The repair shows up on code (+12.2 on our harness) and barely on multiple choice (MMLU and TruthfulQA within noise at $n = 250$). That asymmetry is itself evidence. A truncated program is worthless; a truncated multiple-choice trace often still contains a recoverable letter. **The benchmark that most clearly exposes this failure is the one whose output cannot be salvaged.** Benchmarks that can salvage it will under-report the damage.

5.6 The teacher does it too

In the on-policy experiment of §7.5, the distilled model was rolled out over a fresh 4,974-prompt pool and only its 1,247 failures were sent to the teachers. The 35B code teacher verified on **29%**

Table 5: Budget forcing on the same weights, our harness, reserve $N = 256$. *Plain*: standard decoding under the budget. *Forced*: two-phase decoding; items still inside `<think>` at the phase-1 cap have the block closed for them and receive the reserve to answer. No item receives more than the budget in total.

Model	Benchmark	Budget	Plain	Trunc.	Forced	Forced $\rightarrow$ correct	Δ
v1	HumanEval	2,048	130/164 (79.3)	24	142/164 (86.6)	12 of 29	+12
v1	HumanEval	4,096	138/164 (84.2)	19	144/164 (87.8)	6 of 18	+6
stock oQ4	HumanEval	4,096	118/164 (72.0)	39	<i>pending</i>		
v1	MMLU ₂₅₀	3,072	<i>pending</i>				

of those and **failed to terminate on 38%** even after budget escalation to 4,608 tokens—against 73% verified on random prompts from the same pool under the same teacher and verifier. On the problems a quantized 9B cannot finish, a 4-bit 35B frequently cannot finish either. The failure is not specific to the small model; it is a property of hard problems under a budget, and quantization moves the threshold.

6 Repairing it at decode time: HALT

If the failure is stopping, the cheapest test is to stop the model for it. HALT (Harness-Applied Limit on Thinking) is the closing half of budget forcing [15]: when the reasoning block has not closed by the time the budget is nearly spent, the harness closes it—appending a one-line commit phrase and `</think>`—and allows a reserved number of tokens for the answer. Concretely, decoding runs in two phases: $B - N$ tokens, then N more for anything that hit the first cap, with `</think>` forced only for items still inside the block. No item receives more than B tokens in total. One implementation detail mattered: an earlier version took the reserve out of *every* item’s budget and cut a correct solution mid-answer; the per-item diff caught it where the aggregate did not.

Table 5 and Figure 3 give the result. Two observations stand out. First, **the distilled model under HALT at 2,048 tokens outperforms the same model allowed 4,096 without it**—half the budget, more correct answers. Forcing is not only recovering truncations; it is stopping the model before longer reasoning turns into wrong answers. Second, in a 24-item pilot, a problem the model reasoned on to $\approx 2,050$ tokens and answered *wrong* was answered *right* when cut at 1,792: past some point, continued reasoning has negative value and the model does not detect it. Wall clock is unchanged.

HALT was the ninth thing we tried. It is a two-phase change to the decoder, it generalises to any model with this failure mode including the stock build we would have shipped without distillation, and it needed no training.

7 What did not work, and what the controls established

Table 6 and Figure 4 list every run after v1. Each moved one variable, each had a prediction recorded before measurement, and each was measured on oMLX under the protocol of Table 2. Across the six completed comparable runs, **21 of 24 benchmark deltas against v1 are negative and none is positive beyond noise**. v1 is not merely the best result; it behaves like a sharp optimum that every perturbation moves away from.

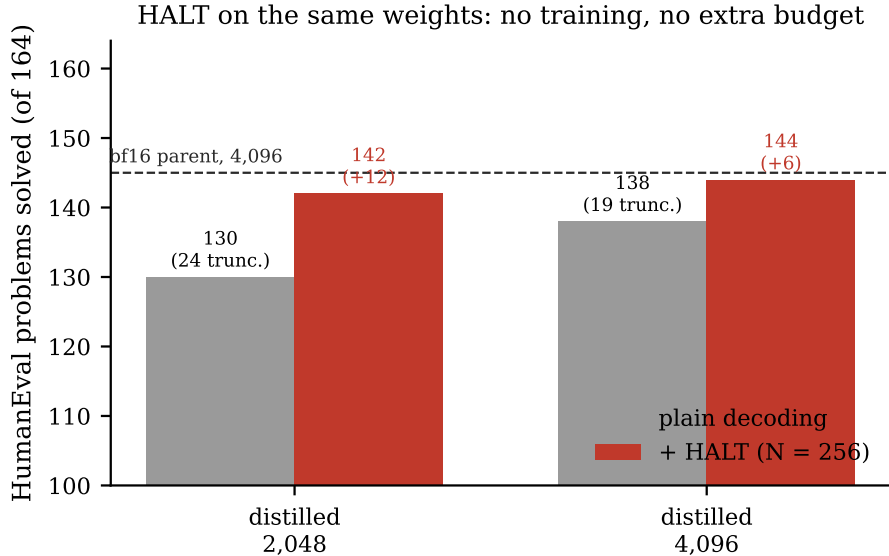


Figure 3: HALT on the same weights (Table 5). Grey: plain decoding under the budget, with the number of problems truncated before answering. Red: two-phase decoding with a 256-token reserve; no item receives more total budget. Dashed: the bf16 parent at 4,096 on the same harness.

7.1 Data composition (v2–v4)

MMLU sat at 83.5, 83.1, 83.6, 83.2 across mixtures differing in source, volume (1,672 to 4,075 samples) and length distribution, and v1 already matches its 35B teacher there. v4 is the cleanest: v1 had trained only on traces under 1,024 tokens—1,758 of median 678, while 1,735 of median 1,449 were never seen—which §5 predicts should matter. Lifting the cap to 2,048 with sample count, steps and every hyperparameter held, and a `max_samples` cap added so the length change could not smuggle in volume, moved nothing. The pre-registered prediction (HumanEval improves most) was wrong. Telling the code teacher to be brief in v2 cost 5.4 points of HumanEval; asking for shorter abstention traces in v3 produced *longer* ones. Brevity instructions do not reliably work on these models.

7.2 Objective: forward KL makes the model unable to stop (v5)

v5 trained against the teacher’s renormalised top-64 next-token distribution (measured to hold 99.93% of its mass) with loss $0.3 \text{ CE} + 0.7 \text{ KL}(p_T \| p_S)$, everything else at v1. It produced the first model *below stock*: 77.2 MMLU in **69,897 seconds against v1’s 13,007** on the same questions—5.4×. It did not become confused; it became unable to stop. This is the mode-covering failure Gu et al. [6] describe: a 9B at 4.7 bits cannot cover a 35B’s low-probability modes, and one of those modes is the teacher’s residual uncertainty about whether to continue reasoning. Three failure modes were ruled out before accepting the result (vocabulary identity across teachers, top- k coverage, span alignment) and a latent batcher bug was found and fixed on the way. v7 later confirmed the collapse is specific to the objective: a different departure from v1 left wall clock normal.

Table 6: Every training run after v1, each moving one variable, each with a prediction recorded before measurement. oMLX, same protocol as Table 2. v5 was abandoned after MMLU ($5.4\times$ v1’s wall clock). Across the six completed comparable runs, 21 of 24 deltas are negative and none is positive beyond noise.

Run	Change from v1	MMLU	TruthfulQA	HumanEval	Δ vs v1
v1	baseline recipe	83.5	79.0	90.8	—
v2	misconception slice replaces TriviaQA; brief code	83.1	76.9	85.4	-0.4 / -2.1 / -5.4
v3	abstention slice added; v1 code traces restored	83.6	77.4	87.2	+0.1 / -1.6 / -3.6
v4	1,024-token length bias removed, all else held	83.2	76.6	89.6	-0.3 / -2.4 / -1.2
v5	objective $\rightarrow$ 0.3 CE + 0.7 KL(teacher student), top-64	77.2	—	—	-6.3 / — / —
v7	adapter rank 32 $\rightarrow$ 64, data byte-identical	83.2	77.6	88.4	-0.3 / -1.4 / -2.4
v8	on-policy: v1’s own failures, continued from v1	81.6	76.3	85.4	-1.9 / -2.7 / -5.4
v8-ctl	same recipe, random prompts (control)	83.2	77.7	86.6	-0.3 / -1.3 / -4.2
v6	filter $\rightarrow$ recover verified abstentions	stopped before training: 22 usable traces, 0 under the cap			

7.3 Adapter capacity (v7)

Every earlier run used rank 32 over the top 16 layers, chosen once and never revisited. Doubling to 86.6M trainable parameters on byte-identical data (sha256-verified) changed nothing: all three deltas inside one standard error, wall clock normal. Rank 64 across all 32 layers does not fit in 64 GB—backprop *depth*, not rank, is what costs. This converts “capacity, not recipe” from an inference reached by eliminating recipes into a measurement.

7.4 The TruthfulQA regression is an abstention deficit the teacher cannot fix (v3, v6)

Per item, v1 matches stock on ordinary TruthfulQA questions (81.3% vs 81.4%) and **loses 11 points on the 117 whose correct answer expresses uncertainty** (65.0% vs 76.1%; 19 lost, 6 gained, $p \approx 0.016$). Rejection sampling keeps only confident correct answers, so the student never sees warranted uncertainty. Adding 527 SQuAD-v2 unanswerable items (v3) moved TruthfulQA *down* 1.6: reading-comprehension abstention is a different skill. Changing the filter instead (v6)—re-asking the teacher, with its confidence made explicit, the 122 open-ended prompts it had answered wrong, and keeping only the traces where it declined—yielded 22 usable traces, of which none fit the training cap. The number that matters is the other one: **on questions it had already answered wrong, invited to say it did not know, Qwen3.6-35B re-asserted a confident answer 82% of the time**. Sequence-level distillation can only transmit what the teacher emits, and this teacher does not emit calibrated uncertainty. v6 was stopped before training on a threshold set in advance.

7.5 A second training pass from the best checkpoint is itself harmful (v8 and its control)

v8 tested on-policy prompt selection in the spirit of Agarwal et al. [1] at the prompt level: roll v1 out over a fresh pool of 4,974 prompts (a different seed, every v1 prompt excluded, code from KodCode since MBPP was exhausted), keep the 1,247 it fails, have the teacher answer only those, train on the 330 that verify, continuing from v1’s fused checkpoint. A control arm trained on 330 *random* prompts from the same pool with everything else identical. **Both arms lost to v1 on all three benchmarks by similar amounts** (Table 6). Since the arms differ only in prompts, the second pass is the damage, not the selection. Every method that starts from v1 must first beat a

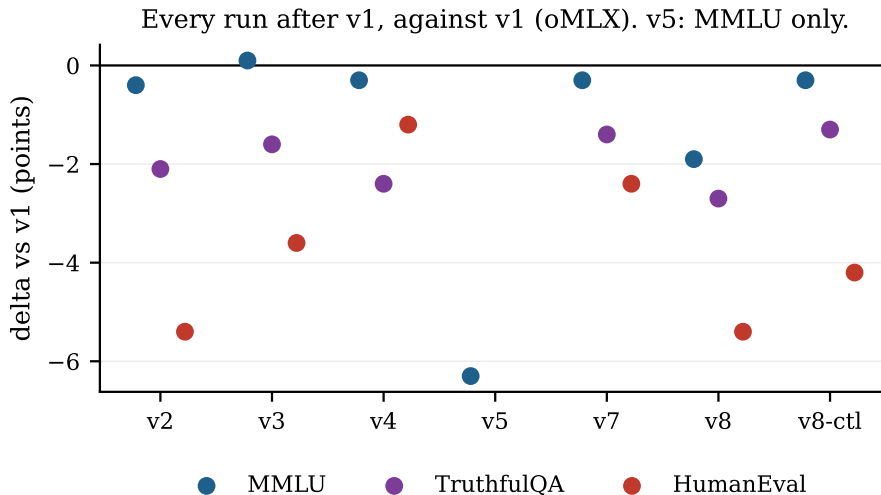


Figure 4: Every completed run after v1, as its delta against v1 on each benchmark (oMLX). Twenty-one of twenty-four deltas are below zero and none is above it beyond noise. v5 was abandoned after MMLU.

1–4 point tax; every method that starts from the base must re-earn v1’s gains. Three bugs surfaced during this run and are fixed in the released code: a verifier ordering that scored model-copied test calls as `NameErrors`, a batching loop that silently dropped prompts at every domain boundary (present since v1), and rollouts that did not store answer text.

8 Engineering: a chunkwise training path for gated DeltaNet

Training was pinned at 1,024-token sequences, which rejected 46% of every trace generated—six times what wrong answers cost. The cause was not the hardware. `mlx_lm` [7] trains GDN layers through a sequential Python loop over every timestep, because the fast Metal kernel used at inference has no gradient; its own docstring calls it a reference implementation for prefill. Measured: ≈ 8 MB of autograd state per token per layer.

The gated delta rule admits a chunkwise-parallel form [20, 19] that keeps $O(T/C)$ materialised states instead of $O(T)$. Two details mattered in MLX. `mx.linalg.tri_inv` is CPU-only with no VJP, so the unit-triangular inverse is built from matmuls by 2×2 block recursion in $\log_2 C$ levels, which autodiff handles natively. And the textbook derivation divides by the cumulative decay, which underflows on strongly decaying heads and sends intermediate writes to infinity; carrying bounded ratios $G_j/G_i \leq 1$ instead is what makes it work on a real model. A fused Metal triangular *solve* with a hand-written VJP then replaces the inverse, which was used exactly once as $A^{-1}b$ and cost 5.07 of the 9.3 ms per chunk.

One GDN layer, forward and backward, cumulative against stock `mlx_lm` at 4,096 tokens: **108.33 GB / 39.02 s $\rightarrow$ 4.60 GB / 0.29 s—24 $\times$ memory, 135 $\times$ speed**. Outputs match the sequential reference to $\sim 3 \times 10^{-7}$ relative in fp32 across all five gradients; the inference kernel matches at 100% top-1 on the full 9B. End-to-end training throughput doubled (55 $\rightarrow$ 111 tokens/s) and 2,048-token training fits in 41.8 GB. Contributed as <https://github.com/ml-explore/mlx-lm/pull/1870> (chunkwise path, five model families), with the fused solve queued behind it.

9 Related work

Distillation objectives. Hinton et al. [9] and Kim and Rush [11] define logit-level and sequence-level distillation respectively; v1 is the latter with a verifier in the loop. Gu et al. [6] argue forward KL is mode-covering and ill-suited to generative distillation; Agarwal et al. [1] add on-policy sampling and generalised divergences; Ko et al. [12] propose skew KL; Shing et al. [17] interpolate the target toward the teacher over training. Our v5 is a direct instance of the forward-KL failure these predict, with termination as the specific casualty.

Test-time compute. Muennighoff et al. [15] introduce budget forcing to *extend* reasoning with “Wait”; we use the closing half of the same mechanism on a quantized model and find that forcing *earlier* than the model would stop itself improves accuracy.

Quantization-aware training and rotation. EfficientQAT [4] and Recover-LoRA [16] train through the quantizer; QuaRot [2] suppresses outliers before it. We trained in bf16 and quantized last, and §7.5’s finding that continued training from the best checkpoint is harmful bears on whether a second, quantization-aware pass could help here.

Linear attention. Yang et al. [20, 19] give the chunkwise delta-rule form §8 implements. Wijmans et al. [18] remove the logit memory that a 248k vocabulary costs and would compound with logit-level distillation. Marathe et al. [14] distil token-level teachers into byte-level students for pretraining; their objective is forward KL, and their setting—a student with no prior distribution to disturb—is one where the failure in §7.2 does not arise.

10 Threats to validity

Scope. One model family, one quantization scheme, one hardware target. The *mechanism* behind the quantized model’s verbosity is documented, not explained: we show that a coarser weight grid produces longer reasoning and that this dominates the measured damage, not why.

Measurement. The quantization-ladder timing (§5) is wall clock, not token counts, because the external harness does not emit them. The two harnesses disagree by 15 points on the stock model’s absolute HumanEval while agreeing on every direction; any single absolute number should be read as approximate. MMLU is a 1,000-question sample; TruthfulQA and HumanEval are complete. Standard errors for the run comparisons are single-proportion bounds (paired errors are tighter), so the “inside noise” claims are conservative.

Unexplained results. In the v8 experiment the random-arm model became 46–80% slower to answer while the failure-arm model barely changed. The failure arm’s traces are by selection the ones a 35B finished cleanly, which may be the one thing error-conditioning did; server load during the control run cannot be excluded. Recorded, not claimed.

What was not run. Token-level on-policy distillation with a mode-seeking divergence and termination-token masking; quantization-aware self-distillation from the bf16 distilled model into its own oq4 build; full-parameter updates via a factored optimiser. §7.5 constrains all three. TruthfulQA has no verifier and its regression was not repaired.

11 Conclusion

A 4-bit reasoning model’s dominant failure is that it does not stop. We found this the slow way: one distillation run that worked, seven that did not, and a control that showed trying harder from the best checkpoint makes it worse. Reading the per-item records and the timing column is what turned the negative results into a diagnosis, and the diagnosis into the one intervention that needed no training. The practical recommendations are short. Report truncation rates next to accuracy. Measure generation length before and after quantizing; wall clock is the cheapest early warning. And treat the token budget as part of the model being evaluated, because for a reasoning model it is.

Reproducibility

Code, configurations, every run’s adapters, fused checkpoints, teacher traces, per-item evaluation records and a 45-section decision log are at https://github.com/adityak74/ornith-1.5-9b-distil_benchmarks/baseline.json holds every number in this paper with its measurement date and harness; `scripts/make_paper_tables.py` regenerates every table in this paper from those files. `odistil eval -force-budget N` reproduces HALT (§6).

References

- [1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. *arXiv preprint arXiv:2306.13649*, 2023. URL <https://arxiv.org/abs/2306.13649>.
- [2] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoeffler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456*, 2024. URL <https://arxiv.org/abs/2404.00456>.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [4] Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, and Ping Luo. Efficientqat: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062*, 2024. URL <https://arxiv.org/abs/2407.11062>.
- [5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023. URL <https://arxiv.org/abs/2305.14314>.
- [6] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*, 2023. URL <https://arxiv.org/abs/2306.08543>.
- [7] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: An array framework for apple silicon. <https://github.com/ml-explore/mlx>, 2023.

- [8] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. URL <https://arxiv.org/abs/1503.02531>.
- [10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- [11] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. *arXiv preprint arXiv:1606.07947*, 2016. URL <https://arxiv.org/abs/1606.07947>.
- [12] Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. Distillm: Towards streamlined distillation for large language models. *arXiv preprint arXiv:2402.03898*, 2024. URL <https://arxiv.org/abs/2402.03898>.
- [13] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2022. URL <https://arxiv.org/abs/2109.07958>.
- [14] Kalyani Marathe, Artidoro Pagnoni, Tomasz Limisiewicz, Margaret Li, Mike Lewis, Luke Zettlemoyer, and Srinivasan Iyer. Breaking the token ceiling: Distilling smaller, stronger byte models. *arXiv preprint arXiv:2609.12303*, 2026. URL <https://arxiv.org/abs/2609.12303>.
- [15] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025. URL <https://arxiv.org/abs/2501.19393>.
- [16] Recover-LoRA authors. Recover-lora: Data-free accuracy recovery of degraded language models via low-rank adaptation. *arXiv preprint arXiv:2510.08600*, 2025. URL <https://arxiv.org/abs/2510.08600>.
- [17] Makoto Shing, Kou Misaki, Han Bao, Sho Yokoi, and Takuya Akiba. Taid: Temporally adaptive interpolated distillation for efficient knowledge transfer in language models. *arXiv preprint arXiv:2501.16937*, 2025. URL <https://arxiv.org/abs/2501.16937>.
- [18] Erik Wijmans, Brody Huval, Alexander Hertzberg, Vladlen Koltun, and Philipp Krähenbühl. Cut your losses in large-vocabulary language models. *arXiv preprint arXiv:2411.09009*, 2024. URL <https://arxiv.org/abs/2411.09009>.
- [19] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024. URL <https://arxiv.org/abs/2412.06464>.
- [20] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024. URL <https://arxiv.org/abs/2406.06484>.